

## **Práctica 02.**

Nombre del Equipo: Las pros y Diego.

Práctica: 02

### **Integrantes:**

- Ciro Ortiz Diego - No. Cuenta: 323238300
- Soto Álvarez Regina - No. Cuenta: 323313117
- Santuario Téllez Melanie Elizabeth - No. Cuenta: 323136385

### **Descripción de la práctica.**

Se implementó una simulación interactiva para un puesto automatizado de pizzas y helados ("El Pequeño Cesarín") atendido por un robot interactivo.

Nuestro sistema permite:

- ★ Atención por estados del Robot: Transicionar al robot entre sus diferentes estados (Dormido, Atendiendo, Preparando y Esperando Entrega).
- ★ Preparación de Pizzas: Configurar el tipo de masa y procesar la elaboración de pizzas siguiendo una secuencia estricta de pasos de preparación.
- ★ Atención por estados del Robot: Transicionar al robot entre sus diferentes estados (Dormido, Atendiendo, Confirmado, Preparando y Esperando Entrega).
- ★ Preparación de Pizzas: Configurar el tipo de masa y procesar la elaboración de pizzas siguiendo una secuencia estricta de pasos de preparación.
- ★ Personalización de Helados: Seleccionar una base de helado con un sabor determinado y agregar múltiples toppings o coberturas de forma dinámica, controlando un límite máximo de 3 porciones por ingrediente.
- ★ Generación de Ticket: Desglosar el detalle de los productos ordenados, la lista de ingredientes adicionales agregados y el total acumulado a pagar. Personalización de Helados: Seleccionar una base de helado con un sabor determinado y agregar múltiples toppings o coberturas de forma dinámica, controlando un límite máximo de 3 porciones por ingrediente.
- ★ Generación de Ticket: Desglosar el detalle de los productos ordenados, la lista de ingredientes adicionales agregados y el total acumulado a pagar.

### **Patrones aplicados.**

State:

Se eligió el patrón State para modelar el comportamiento del robot cajero/cocinero. Puesto que las acciones del robot dependen enteramente del estado en el que se encuentra actualmente, este patrón evita estructuras condicionales compuestas (if-else o switch). Permite delegar la responsabilidad de cada acción a clases concretas (EstadoDormido, EstadoAtendiendo, EstadoConfirmado, EstadoPreparando, EstadoEsperandoEntrega), logrando un código limpio y extensible.

### Template:

Se aplicó Template Method en la jerarquía de Pizza. La preparación de cualquier pizza requiere una serie de pasos invariables y en un orden predefinido (preparar masa, aplanar, poner salsa, hornear, empaquetar), mientras que únicamente varían ciertos ingredientes como el tipo de queso o la proteína. La clase abstracta Pizza define el método plantilla prepararPizza() fijando el algoritmo, permitiendo que las subclases concretas (PizzaCarnivora, PizzaHawaiana, PizzaPepperoni, etc.) sobrescriban únicamente los pasos específicos.

### Decorator:

Se implementó el patrón Decorator para la estructura de Helado. Puesto que un cliente puede combinar una base de helado con una cantidad indeterminada de ingredientes extra (como chispas, gomitas o fruta) en tiempo de ejecución, la herencia tradicional provocaría una explosión de clases. Con Decorator, la clase abstracta IngredienteExtra envuelve dinámicamente cualquier objeto de tipo Helado, permitiendo acumular precios y descripciones de forma flexible sin alterar la clase base HeladoBase.

### Versión de Java.

- Versión de Java utilizada: JDK 23

### Instrucciones de compilación y ejecución.

Para compilar (desde la carpeta src/):

```
rm -rf bin && mkdir bin && javac -d bin *.java
```

En Windows (CMD / PowerShell desde la carpeta src/):

Compilar (CMD):

```
if exist bin rmdir /s /q bin && mkdir bin && javac -d bin *.java
```

Compilar (PowerShell):

```
if (Test-Path bin) { Remove-Item -Recurse -Force bin }; New-Item -ItemType Directory -Name bin; javac -d bin *.java
```

Para ejecutar (desde la carpeta src/):

```
java -cp bin Main
```

### Observaciones adicionales.